

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	B. Manu Jaswanth	Reg. No.:	192525400
Section / Batch:	2025	Date:	16/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Application-Based Questions	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS**Q1. Big Data Analytics Pipeline**

A data analytics pipeline processes input using $T(n) = T(n-1) + \log n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive complexity. Analyze how the algorithm scales. Interpret efficiency using asymptotic notation.

Q2. Search Engine Indexing

A search engine indexing algorithm is defined as: $T(n) = 2T(n/2) + n^2$. Apply the Master Theorem and determine the time complexity. Clearly explain the chosen case. Analyze how indexing time grows with web pages.

Code of Conduct

I B. ManuJaswanth (192525400) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **manu**

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (50 Marks)						
Q2 (50 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. A data analytics pipeline processes input using $T(n) = T(n-1) + \log n$. Apply the substitution method to solve the recurrence. Expand step-by-step and derive complexity. Analyze how the algorithm scales. Interpret efficiency using asymptotic notation.

ANSWER:

Big Data Analytics Pipeline

Given Recurrence

$$T(n) = T(n-1) + \log n$$

Step 1: Apply the Substitution Method

Expand the recurrence repeatedly.

$$T(n) = T(n-1) + \log n$$

Substitute $T(n-1)$:

$$\begin{aligned} T(n) &= [T(n-2) + \log(n-1)] + \log n \\ &= T(n-2) + \log(n-1) + \log n \end{aligned}$$

Again,

$$= T(n-3) + \log(n-2) + \log(n-1) + \log n$$

Continuing,

$$= T(1) + \sum_{i=2}^n \log i$$

Step 2: Simplify the Summation

Using the logarithm property,

$$\begin{aligned} \sum_{i=2}^n \log i &= \log(2 \times 3 \times 4 \cdots n) \\ &= \log(n!) \end{aligned}$$

Using **Stirling's Approximation**,

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Taking logarithm,

$$\log(n!) = \Theta(n \log n)$$

Therefore,

$$T(n) = T(1) + \Theta(n \log n)$$

Since $T(1)$ is constant,

$$\boxed{T(n) = \Theta(n \log n)}$$

Step 3: Asymptotic Notation

Big-O

$$T(n) = O(n \log n)$$

Upper bound of running time.

Big-Theta

$$T(n) = \Theta(n \log n)$$

Exact running time.

Big-Omega

$$T(n) = \Omega(n \log n)$$

Lower bound of running time.

Step 4: Complexity

$$\Theta(n \log n)$$

Step 5: Scalability Analysis

- Every new data item requires an additional **log n** processing time.
- Total work grows as **n log n**.
- Growth is much slower than **n²**.
- The pipeline remains efficient for very large datasets.

Step 6: Interpretation

In a Big Data Analytics Pipeline, each incoming record requires logarithmic processing such as searching, balancing, or indexing. Since all records are processed, the total execution time becomes **$\Theta(n \log n)$** , making the pipeline scalable for large datasets.

Q2. A search engine indexing algorithm is defined as: $T(n) = 2T(n/2) + n^2$. Apply the Master Theorem and determine the time complexity. Clearly explain the chosen case. Analyze how indexing time grows with web pages.

Given Recurrence

$$T(n) = 2T\left(\frac{n}{2}\right) + n^2$$

Step 1: Identify Master Theorem Parameters

General form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Here,

$$a = 2$$

$$b = 2$$

$$f(n) = n^2$$

Step 2: Compute

$$n^{\log_b a} = n^{\log_2 2} = n^1 = n$$

Thus,

$$f(n) = n^2$$

Compare:

$$n^2 > n$$

Step 3: Choose the Correct Master Theorem Case

Since

$$f(n) = \Omega(n^{1+\varepsilon})$$

where

$$\varepsilon = 1$$

and the regularity condition holds,

$$2\left(\frac{n}{2}\right)^2 = \frac{n^2}{2} \leq cn^2$$

for some constant

$$c < 1$$

Therefore,

Master Theorem Case 3 applies.

Step 4: Final Complexity

According to Case 3,

$$T(n) = \Theta(n^2)$$

Step 5: Asymptotic Notation

Big-O

$$O(n^2)$$

Big-Theta

$$\Theta(n^2)$$

Big-Omega

$$\Omega(n^2)$$

Step 6: Growth Analysis

- The indexing work doubles with recursive calls.
- Each level performs n^2 additional work.
- The total execution time grows quadratically.

Step 7: Interpretation

In a search engine, webpages are recursively divided into smaller groups for indexing. However, combining and processing the indexed data requires n^2 work at each level. Therefore, the overall indexing algorithm runs in $\Theta(n^2)$ time, meaning indexing becomes significantly slower as the number of webpages increases.